Retrieval-Augmented Generation for Nepali Government Documents

A Hybrid Pipeline for Low-Resource, Code-Switched Legal Text

ADHIKARI PANKAJ

Student ID: 2120256037 ▪ Masters Student

Nankai University

August 27, 2026

Introduction: What is RAG?

Retrieval-Augmented Generation (RAG) — in plain language

Instead of asking an AI to *remember* laws, we first **search official PDFs** for the right paragraph, then let the AI **read that paragraph** to answer.

Why it matters

- Kathmandu Valley bylaws are in Nepali PDFs
- Citizens mix Nepali + English when asking questions
- General chatbots lack your local municipal files

Our focus

- Municipal bylaws & federal laws
- Local, offline-friendly retrieval
- Measuring when RAG beats direct LLMs

Problem Statement

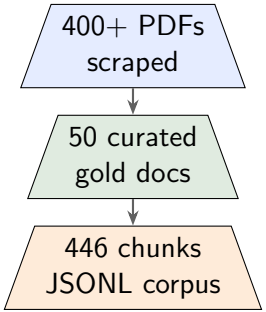
Three barriers for Nepali legal RAG

1. **Legacy PDF encoding** — many files use old fonts; extracted text looks like gibberish, not Unicode Nepali.
2. **Code-switched queries** — users ask in mixed Nepali/English; English-only embeddings struggle.
3. **Evaluation traps** — wrong chunk IDs can make good retrieval look like 0% accuracy.

Example corruption:

`sf7df8f}F dxfgu/kflnsf` → **should be readable Nepali municipal text**

Dataset & Pipeline Overview



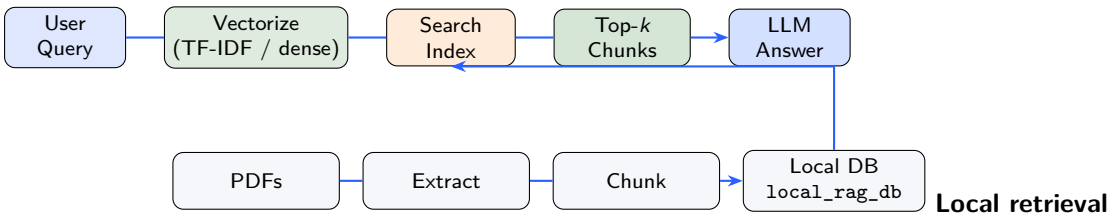
Sources: Kathmandu, Lalitpur, Bhaktapur + federal laws

Categories: bylaws, admin procedures, financial schedules

Pipeline stages

1. Scrape municipal law portals (Python)
2. Curate manifest (`curated_corpus_manifest.jsonl`)
3. Hybrid extract: PyMuPDF + Surya OCR
4. Semantic chunking → JSONL
5. Retrieve → answer with citations

RAG Architecture



database (`local_rag_db/documents.jsonl`) stores chunked text plus optional normalized aliases for corrupted PDFs.

Methodology

Extraction & chunking

- **Router:** text PDFs → PyMuPDF; scanned → Surya OCR (MPS)
- **Chunking:** heading-aware sections; 512-token fallback
- **Logging:** per-document metrics in JSONL

Tools

- Python, regex, JSON/JSONL
- LangChain Core, scikit-learn
- FAISS, sentence-transformers

Hybrid retrieval (our fix)

- **Metadata aliasing:** inject readable Nepali/English keywords into chunk text
- **Sparse TF-IDF:** word + bigram vectors (keyword matching)
- **Dense baseline:** BGE-M3, MiniLM via FAISS

Terminology note

We use *TF-IDF sparse retrieval* in code; BM25 is the same family of keyword methods (not separately implemented).

Literature Review (Brief)

- **Low-resource NLP:** English-heavy models underperform on Devanagari and code-switching.
- **South Asian digitization:** legacy font encodings break standard OCR and embeddings.
- **Hybrid retrieval:** combining sparse (keyword) and dense (semantic) search improves robustness when text is noisy.
- **Legal IR:** answers often require exact tables, fees, and document lists — retrieval must be precise.

Aim & Objectives

Primary aim

Build and evaluate a **local RAG pipeline** that retrieves correct clauses from Nepali municipal PDFs using realistic code-switched queries.

Objectives

1. Curate and extract a reproducible legal corpus (50 PDFs $\rightarrow$ JSONL chunks).
2. Measure dense embedding baselines (BGE-M3, MiniLM) on raw extracted text.
3. Design alias-augmented sparse retrieval and compare Recall@k / MRR.
4. Contrast **grounded RAG answers** vs. **direct LLM guesses** on tax and fee queries.

Understanding the Metrics

Recall@k (we report @3 and @5)

Did the **correct chunk** appear in the top k results?

- 1.0 = found in top-k
- 0.0 = not found

Critical for RAG: if retrieval misses, the LLM hallucinates.

MRR (Mean Reciprocal Rank)

How high is the correct chunk ranked?

- Rank 1 $\rightarrow$ score 1.0
- Rank 2 $\rightarrow$ 0.5
- Rank 3 $\rightarrow$ 0.33

Higher MRR = less noise for the LLM to filter.

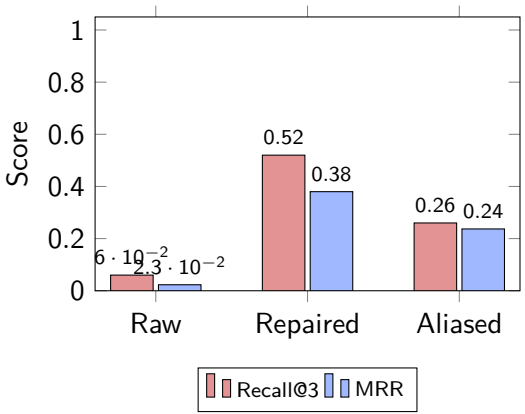
Results: Why Raw Retrieval Fails

Sparse TF-IDF on *raw* extracted text:

- Recall@3 = **0.060**, MRR = **0.023**, doc-level **0.140**
- Cause: 26/50 documents extract as Preeti font gibberish
- Every retriever is broken on raw text — both dense encoders score ≤ 0.040

Key result — deterministic Preeti $\rightarrow$ Unicode repair: Recall@3 rises to **0.520** (doc-level **0.800**) with *no* hand-authored aliases.

Results: Retrieval Performance



System	R@3	MRR	DocR@3
Raw corrupted	0.060	0.023	0.140
Preeti repair	**0.520**	**0.380**	**0.800**
Aliasing (pilot)	0.260	0.237	0.480

Repair **beats** hand-authored aliasing on the leakage-free 50-query bank.

Key Finding: Direct LLM vs. Local RAG

Query	Direct LLM	Our local RAG
Kathmandu parking fee (two-wheeler)	**Guess:** "Usually Rs 20/hr — check website."	**From PDF:** Retrieves `KTM_FIN_002` parking fee clause.
Property tax 1–2 crore slab	**Missing:** "I don't have exact local slabs."	**From PDF:** Exact rate from tax schedule chunk.
Bhaktapur map-pass documents	**Generic:** citizenship, blueprint, tax receipt	**From PDF:** Full checklist from by-law section.

RAG returns **citable, numerically exact** values from source PDFs; general LLMs **hallucinate or generalize** local fees.

Conclusion & Future Work

What we showed

- Built end-to-end pipeline: scrape → curate → extract → chunk → retrieve
- Raw retrieval fails on corrupted Nepali PDF text (Recall@3 = 0.060, 50-query bank)
- **Deterministic Preeti → Unicode repair** lifts Recall@3 to **0.520** (doc-level 0.800) with no hand-authored aliases
- Repair *beats* hand-authored aliasing (0.260) and is necessary-but-not-sufficient for dense retrieval
- Local RAG grounds answers in municipal PDFs; direct LLMs do not

Next steps

- Learned transliteration repair for lossy conjuncts; re-run BGE-M3 fairly
- Add BM25 + dense fusion; expand human-labeled query bank
- Deploy citizen-facing chatbot with mandatory citations

Thank You

Questions?

Repository: `scrapktm/thesis/` | Slides: `presentation/presentation.pdf`